CHAPTER 16

The Messages API

In this chapter

Introduction: The Agent That Ate Its Own Tool Calls . . . 3
16.1 One Endpoint . . . 4
16.2 The Request Body . . . 4
16.3 system Is a Field, Not a Message . . . 5
16.4 Messages . . . 6
16.5 Content Is an Array of Typed Blocks . . . 6
16.6 The Response . . . 7
16.7 stop_reason Is the Control-Flow Branch . . . 7
16.8 Stop Reasons Are Not Errors . . . 8
16.9 Statelessness, and What It Costs . . . 9
16.10 Prefilling Is Gone . . . 9
16.11 Images, Briefly . . . 10
16.12 Two Ways to Build . . . 10
16.13 Visual Analogies . . . 11
16.14 Architecture: One Adapter, Not Fifty Call Sites . . . 11
16.15 Examples: Four Requests . . . 12
16.16 Code Walkthrough: The Protocol, Not the Wrapper . . . 13
16.17 Mini Project: A Client Without an SDK . . . 14
16.18 Panel A — The Wire . . . 15
16.19 Panel B and C — The State Machine, and the Bug . . . 16
16.20 Panel D — Statelessness Is Quadratic . . . 16
16.21 Panel E — The Same Conversation, Cached . . . 17
16.22 Panel F — Six Requests the Validator Refuses to Send . . . 18
16.23 Evaluation and Deployment . . . 18
16.24 Hands-on Labs . . . 19
16.25 Lab Solutions . . . 19
16.26 Best Practices . . . 20
16.27 Common Mistakes . . . 20
16.28 Performance Considerations . . . 22
16.29 Security Considerations . . . 22
16.30 Real Production Examples . . . 23

16.31 Certification Notes 23
16.32 Twenty Practice Questions 24
16.33 Answers and Explanations 25
16.34 Ten Practical Exercises 26
16.35 Five Challenge Labs 27
16.36 Ten Interview Questions 27
16.37 Summary 27
16.38 Glossary 29
16.39 Further Reading 29

Part 3 of 13 · Chapter 16 of 64. Next: Chapter 17 — Streaming, Vision, Files and Documents.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Construct a Messages API request by hand — endpoint, headers, required and optional fields — without an SDK.
2. Explain why `system` **is a top-level field rather than a message**, and describe the one documented exception and why it exists.
3. Read a response as an **array of typed content blocks**, and say why `content[0].text` is a bug rather than a shortcut.
4. Name all seven `stop_reason` values, say which two are non-terminal, and write the control-flow branch each one requires.
5. Distinguish a **stop reason from an error**, and explain why a refusal arrives as an HTTP 200.
6. Explain what **statelessness** costs, quantify its quadratic growth over a conversation, and price the fix.
7. State what changed about **prefilling** and what replaced it.
8. Choose between the **Messages API** and **Claude Managed Agents** for a described workload.

NOTE

Prerequisites

Chapter 1 for the statelessness of inference, which this chapter measures in tokens and pounds. **Chapter 6** for why a token is not a word and why `count_tokens` is the authority. **Chapter 11** for the rate card and the cache multipliers. **Chapter 13** for the `thinking` and `output_config` fields that appear in the request body. **Chapter 14** for the model IDs and the per-model rules the request validator enforces. **Chapter 15** for how you chose the model you are about to name.

Chapters 14 and 15 produced a model. This chapter is the request that uses it. The API surface here is deliberately the *core* of the endpoint: streaming is Chapter 17, thinking as an API surface is Chapter 18, tools are Part 6, and errors and retries are Chapter 29. What is here is the shape everything else hangs on.

Introduction: The Agent That Ate Its Own Tool Calls

An agent is deployed. It works, mostly. Roughly one request in eight produces a reply that is oddly generic — correct in tone, empty of content, as though the model had forgotten what it was asked. Nobody can reproduce it reliably. The logs show HTTP 200 and a normal-looking latency, so the monitoring is clean.

The bug is one line, and it is the first line most people write:

```
answer = response["content"][0]["text"]
```

On a simple text reply that line is correct. On a response where Claude has decided to call a tool it raises a `KeyError`, because block zero is a `tool_use` block with no `text` field. On a 5-series model with thinking on by default, block zero is a `thinking` block — also no `text` field. On a refusal, `content` is empty and the line raises `IndexError`. And in the cases where it *does* return a string, it returns the first text block only, so a response that put a sentence before a tool call returns the sentence and silently drops the call.

Run it against every documented stop reason and the arithmetic is stark: `content[0].text` **raises on three of the seven, and returns a partial answer with no warning on two more.** Section 16.19 prints the table.

The team's fix was a `try/except`. The actual fix is a change of model — not the language model, the mental model. `content` is an array of typed blocks and always was. `stop_reason` is a control-flow branch and always was. `system` is a field, not a message. Statelessness means the conversation is a list you resend, and the cost of that is quadratic. None of these are surprises hidden in a corner of the documentation; they are the first paragraph of it. They surprise people because an SDK makes the easy case so easy that the shape never has to be learned.

This chapter teaches the shape, by building a client without an SDK. Not because you should ship one — you should not, and section 16.14 says why — but because the fastest way to stop being confused by a wrapper is to have written the thing it wraps.

16.1 One Endpoint

Everything in Part 5 of this book, every tool call in Part 6, every agent loop in Part 9 goes through one HTTP request:

```
POST https://api.anthropic.com/v1/messages
x-api-key: sk-ant-...
anthropic-version: 2023-06-01
content-type: application/json
```

Three headers. The API key authenticates you; Chapter 26 covers key management and the alternatives. `anthropic-version` pins the API's own contract and is **required** — it is not the model version, and it has been `2023-06-01` throughout the life of this book's subject matter. `content-type` is ordinary JSON.

A fourth header appears whenever you use a preview feature: `anthropic-beta`, carrying a comma-separated list. Chapter 13's `task-budgets-2026-03-13` and Chapter 14's `output-300k-2026-03-24` both arrive this way.

That is the entire transport. Everything else is the body.

16.2 The Request Body

Three fields are required and the rest are optional.

Field	Required	What it is
`model`	**yes**	A model ID from Chapter 14. A pinned snapshot.
`max_tokens`	**yes**	Hard ceiling on generated tokens, thinking included.
`messages`	**yes**	The conversation so far, as an array.
`system`	no	Instructions with operator authority. A **top-level field**.
`tools`	no	Tool definitions. Part 6.

Field	Required	What it is
`thinking`	no	`{"type": "adaptive"}` or the legacy `enabled` form. Ch 13.
`output_config`	no	`effort`, `task_budget`, structured-output format. Ch 13.
`stop_sequences`	no	Strings that end generation when emitted.
`stream`	no	Server-sent events instead of one response. Ch 17.
`metadata`	no	Caller-supplied identifiers for your own accounting.

`max_tokens` deserves a moment because it is the only *enforced* limit in the whole configuration surface. Chapter 13 established that `effort` is a behavioural signal and a task budget is advisory; `max_tokens` is neither. It truncates, it counts thinking tokens, and it must fit inside the model's output ceiling — 128K on the 5-series, 64K on Haiku 4.5, with Chapter 14's batch exception.

One deprecation, because it will be in code you inherit. `temperature`, `top_p` **and** `top_k` **are not supported on Claude 4.7 and later models and return a 400 when set to a non-default value.** The documented replacement is to omit them and steer with prompting. They remain in the SDK request types, so this fails at runtime and never at compile time — which is why Chapter 14's preflight exists.

16.3 `system` Is a Field, Not a Message

This is one of the book's running threads and it is worth being precise about why.

> DEFINITION
>
> **The system prompt**
>
> The **system prompt** is a top-level `system` field, a sibling of `messages` rather than an entry inside it. It carries **operator authority**: Chapter 7's constitution names Anthropic, operators and users as the three principals, and the system prompt is where operator instructions live. *You are the operator, and operator authority lives in the system prompt.*

Passing it as a `{"role": "system", ...}` message at the start of `messages` is the commonest shape error in the ecosystem, mostly because a different vendor's API works that way. Section 16.22 shows the validator refusing it.

But the picture gained a nuance in 2026 and repeating the old rule without it is now wrong.

> WARNING
>
> **Mid-conversation system messages**
>
> On **Claude Fable 5, Claude Mythos 5, Claude Opus 4.8 and Claude Opus 5** you *can* include a message with `"role": "system"` **after a user turn**, to add an instruction partway through a conversation. Two rules govern it:
>
> - **It cannot be the first entry in `messages`.** Instructions that apply from the start belong in the top-level `system` field.
> - **It carries the same authority as the top-level field.**
>
> The reason it exists is a caching one, and it is elegant. The top-level `system` field sits at the *front* of the rendered prompt, so changing it invalidates every cached prefix behind it — Chapter 11's arithmetic, applied to instructions. A mid-conversation system message is *appended at the end*, so it adds authority without invalidating anything in front of it.
>
> The design rule follows directly: **top-level `system` for instructions that apply from the very first turn; a mid-conversation system message for instructions that only become relevant later.**

Note who is on that list and who is not. Sonnet 5 is not, and neither is Haiku 4.5. Chapter 14's rule holds: capability does not travel in a bundle, and the validator in section 16.22 checks the model before it accepts the message.

16.4 Messages

`messages` is an array of turns. Each turn has a `role` and a `content`.

The first message must be `user`. A conversation begins with the person, not the assistant.

Roles alternate, user and assistant, and the array is the whole history.

Earlier turns need not have come from Claude. The documentation is explicit that you can use **synthetic assistant messages** — turns you wrote yourself and are presenting as though the model had said them. This is a genuinely useful technique for few-shot examples and for steering, and it is also a reminder of where the trust boundary sits: the model has no way to distinguish a turn it produced from one you fabricated. Chapter 58 makes the security argument; the mechanical point here is simply that the history is yours to construct.

A tool-use loop is still one assistant turn. Chapter 13 established this and it matters for the request shape: when Claude stops with `tool_use`, the continuation is a `user` message containing **nothing but `tool_result` blocks**, one for each `tool_use` block. Adding text after them ends the turn, and for a server tool that was left waiting it produces a 400. Part 6 builds this properly.

16.5 Content Is an Array of Typed Blocks

The other running thread, and the one the introduction's bug violated.

```
"content": [
  { "type": "thinking", "thinking": "", "signature": "Eos..." },
  { "type": "text", "text": "Let me look that up. " },
  { "type": "tool_use", "id": "toolu_01", "name": "get_weather",
```

```
    "input": { "location": "Leeds" } }
]
```

Block types you will meet in this book: `text`, `thinking` and `redacted_thinking` (Chapter 13), `tool_use`, `tool_result`, `server_tool_use` and `mcp_tool_use` (Part 6), `image` and `document` (Chapter 17), `search_result` and citation-bearing blocks (Chapter 8).

Two conveniences that hide the shape and are worth understanding rather than avoiding. On the **request** side you may pass `content` as a plain string, which is expanded to a single `text` block — the harness in section 16.16 does the expansion explicitly so you can see it happen. On the **response** side there is no such convenience: `content` is always an array, and the correct read is to filter by type.

KEY TAKEAWAYS

The correct read

```
text = "".join(b["text"] for b in content if b["type"] == "text")
```

Filter by type; never index by position. Block zero is a `thinking` block on every 5-series model by default, a `tool_use` block whenever Claude calls a tool, and absent entirely on a refusal.

16.6 The Response

```
{
  "id": "msg_01XFDUDYJgAACzvnptvVoYEL",
  "type": "message",
  "role": "assistant",
  "model": "claude-opus-5",
  "content": [ { "type": "text", "text": "Hello!" } ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "stop_details": null,
  "usage": { "input_tokens": 12, "output_tokens": 6 }
}
```

Field by field. `id` identifies the message and belongs in your logs — Chapter 14 argued that the model ID and the response ID are what make a decision defensible after a model is retired. `model` echoes back the model that served the request, which is worth asserting against what you asked for. `content` is the block array. `stop_reason` is section 16.7. `stop_sequence` names which of your stop sequences fired, and is `null` otherwise. `stop_details` is `null` for every stop reason except `refusal`, where it identifies the policy category. `usage` carries `input_tokens` and `output_tokens`, plus the cache fields of Chapter 11 and, from Chapter 13, `output_tokens_details.thinking_tokens`.

`usage` is not diagnostic detail. It is your bill, arriving with every response, and a system that does not log it cannot answer any question in Chapter 15.

16.7 `stop_reason` Is the Control-Flow Branch

Seven values. Two of them mean the turn is not finished.

stop_reason	Terminal?	What you must do
`end_turn`	yes	Use the response.
`max_tokens`	yes	Truncated. Raise the limit or continue the response.
`stop_sequence`	yes	Read `stop_sequence` to see which fired.
`tool_use`	**no**	Run the tool; return `tool_result` blocks.
`pause_turn`	**no**	Send the assistant content back unchanged.
`refusal`	yes	Read `stop_details`; retry on a fallback model.
`model_context_window_exceeded`	yes	Treat the response as truncated.

Notes on the four that are not obvious.

`tool_use` means Claude has stopped mid-turn awaiting you. A response can also contain a `server_tool_use` block whose `id` has no matching result block, meaning that server tool has not run yet — commonly because Claude called a server tool and one of your client tools in the same batch, and the API returned so you could run yours first. There is no separate marker; you detect it by checking each `server_tool_use` or `mcp_tool_use` id for a matching result block.

`pause_turn` is different and people conflate the two. It means the *server-side* sampling loop hit its iteration limit while executing server tools — the default is **10 iterations per request**. The continuation is not tool results; it is the assistant content sent back as-is. A response waiting on *your* tools never has `stop_reason: "pause_turn"`.

`refusal` arrives as a normal **HTTP 200**, not an error. `stop_details` carries the policy category. The documented remedy is a retry on a fallback model, and Chapter 29 covers the retry machinery.

`model_context_window_exceeded` means generation filled the context window rather than your `max_tokens`. It exists so you can request the maximum possible output without computing your input size first. It is typed in the SDKs' beta namespace; on Sonnet 4.5 and newer the API returns it without a beta header, and earlier models need `model-context-window-exceeded-2025-08-26`.

> WARNING
>
> **An eighth value will appear**
>
> Seven is the count on 29 July 2026. New values ship. A handler with an `else` branch that returns text will one day treat an unfinished turn as a finished one, silently. The harness in section 16.16 raises on an unrecognised `stop_reason` instead, on the principle that a loud failure in staging beats a quiet one in production — and there is a test asserting it.

16.8 Stop Reasons Are Not Errors

Worth its own section because conflating them produces both false alarms and missed failures.

	Stop reason	Error
Where it lives	The response body	The HTTP status and an error body
What it means	Generation ended normally, for a reason	The request could not be processed
HTTP status	200	4xx or 5xx
Content	Valid	An error object
Handling	A branch in your control flow	Retry, back off, or fix the request

A refusal is a successful request whose answer is "no". A 429 is a failed request. Alerting on refusals as though they were outages produces noise; treating `max_tokens` as success produces truncated documents that nobody notices. Chapter 29 is the error half; this chapter is the branch half.

16.9 Statelessness, and What It Costs

Chapter 1's first running thread: inference is forward-only and the server keeps nothing between requests. The documentation states it plainly — *the Messages API is stateless, which means that you always send the full conversational history to the API.*

The consequence is arithmetic that surprises people, so here it is before the measurement. If a turn is `m` tokens, then turn `i` carries `i·m` tokens of history, and over `n` turns the total input is:

```
total  =  m · (1 + 2 + 3 + ... + n)  =  m · n(n+1)/2
```

Quadratic. The naive estimate — "we send about `m` tokens per turn, so `n` turns is `n·m`" — is wrong by a factor of `(n+1)/2`. At thirty turns that is **fifteen and a half times**, and section 16.20 measures **26.2×** on a realistic conversation because the assistant turns are longer than the user turns.

Two corollaries, both of which shape architecture.

Conversation length is a cost decision, not a UX one. Doubling the number of turns quadruples the input bill. Part 10 is entirely about this; Chapter 54's compaction and summarisation are the structural answer.

Prompt caching is the mechanical answer, and it fits this shape unusually well. Chapter 11 established that caching is prefix-only. A conversation is *all prefix*: everything except the newest turn is byte-identical to the last request. Reading it at 0.1× instead of 1× is exactly the right lever, and section 16.21 measures an **82.6% saving at thirty turns**, rising with length.

16.10 Prefilling Is Gone

Prefilling meant putting the opening of the assistant's reply into the last message, so the model continued from it — `{"role": "assistant", "content": "The answer is ("}` to force a single letter out of a multiple-choice question.

> WARNING
>
> **Prefill returns a 400 on Claude 4.6 and later**
>
> **Prefilling is not supported on Claude 4.6 and later models, nor on Claude Mythos Preview.** Requests using it return a 400 error. The documented replacements are **structured outputs** on models that support them, or **system prompt instructions**.

This supersedes the narrower rule that you cannot prefill while thinking is on. Both are true; the model-level rule is stronger and is the one to remember. Chapter 14's preflight check fires on the same requests either way, but for the better reason.

What replaced it is a genuine improvement rather than a workaround. Chapter 10's structured outputs guarantee schema compliance through constrained decoding rather than nudging the model with an opening bracket, and Chapter 23 builds JSON contracts on that guarantee. Prefill was a hack that worked; constrained decoding is a mechanism that holds.

16.11 Images, Briefly

Claude reads text and images in the same request. An image is a content block with a `source`, and there are three source types: `base64`, `url`, and `file`, the last referencing an upload through the Files API. Supported media types are `image/jpeg`, `image/png`, `image/gif` and `image/webp`.

```
{ "type": "image",
  "source": { "type": "url",
              "url": "https://example.com/diagram.png" } }
```

That is the whole shape as far as this chapter goes. **Chapter 17** covers vision properly — resolution, token cost, multi-image requests, documents and the Files API — because the interesting questions are about tokens and quality rather than syntax.

16.12 Two Ways to Build

The documentation opens by naming a fork in the road, and it belongs in a chapter about the Messages API because it is the choice of whether to use it at all.

	Messages API	Claude Managed Agents
What it is	Direct model prompting access	A pre-built, configurable agent harness running in managed infrastructure
Best for	Custom agent loops and fine-grained control	Long-running tasks and asynchronous work

The Messages API gives you the loop. You own the retries, the tool execution, the conversation state, the compaction and the observability — which is most of Parts 9 through 12 of this book, and it is the right choice when the loop is your product or when your requirements are specific. Managed Agents run the harness for you, which is the right choice when the loop is undifferentiated work and the task is long-running or asynchronous.

This book teaches the Messages API, because the certification is about architecture and you cannot architect a loop you have never held. But "build it yourself" is a decision with a cost, and a chapter that pretended otherwise would be selling you something.

16.13 Visual Analogies

VISUAL ANALOGY

A postcard, not a phone call

Every request is a postcard. It carries everything the recipient needs, because they remember nothing about the last one. There is no line held open, no session, no "as I was saying". The conversation exists only in the pile of postcards you keep and recopy in full each time.

Where it breaks. A postcard has a fixed size and you would simply write less; the API charges you for the recopying, and the total grows with the square of the number of cards. Prompt caching is the postal service offering to remember the parts of the card that have not changed and charging a tenth for them — which has no postal analogue at all, and is why Chapter 11's cache multipliers are the most important line on the rate card for conversational workloads.

VISUAL ANALOGY

A returns form with a reason code

`stop_reason` is the reason code on a delivery that came back. "Address unknown", "recipient absent", "refused at the door", "parcel too large" all arrive through the same slot and look identical if you only check that something arrived. What you do next depends entirely on the code, and two of the seven codes mean the delivery is still in progress and the courier is waiting on you.

Where it breaks. Reason codes are stable and this list is not; an eighth will ship. A handler whose default branch assumes "finished" will one day treat an unfinished turn as a finished one, which is why the harness raises on an unknown value rather than falling through.

VISUAL ANALOGY

A shipping manifest, not a parcel

A response is a manifest listing several items by type, not a single wrapped parcel. Reaching in and taking the first item works fine on a shipment that contains one thing, and fails the moment the shipment contains a packing note, an item, and a customs form — which is exactly what a thinking block, a text block and a tool-use block are.

Where it breaks. Manifests are ordered by convention and this one is ordered by generation: thinking first because it was generated first, text and tool calls in the order they were produced. So position does carry meaning; it just does not carry the meaning `content[0]` assumes. Filter by type and the ordering stops mattering.

16.14 Architecture: One Adapter, Not Fifty Call Sites

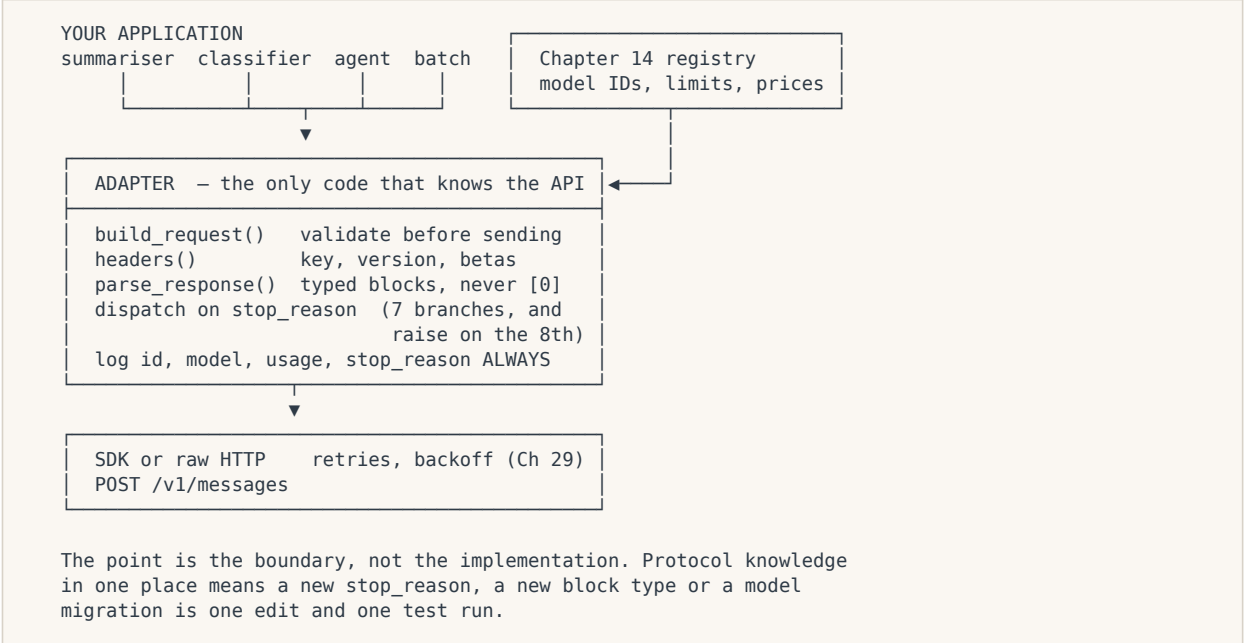


Figure 16.1 — Where the protocol knowledge should live.

Three properties worth designing for.

Use the SDK underneath. This chapter builds a client from scratch to teach the shape; shipping one is a different decision, and the SDKs handle retries, streaming, connection reuse and the client-side rules — such as requiring streaming above `max_tokens` of 21,333 — that you would otherwise rediscover. Build the adapter, not the transport.

Validate before you send. Every check in section 16.22 costs microseconds and saves a round trip and a confusing error at the wrong moment. Chapter 14's preflight is the model-aware half of the same idea.

Log `id`, `model`, `usage` and `stop_reason` on every response, always. These four fields answer "what did we send, what did it cost, and why did it stop" — which is every question Chapters 15, 19, 29 and 57 will ask. They are free and they are gone if you do not capture them.

16.15 Examples: Four Requests

The minimum. `model`, `max_tokens`, one user message. Three fields, and it works. Everything else in this chapter is optional, which is worth knowing when you are debugging: strip back to this and add one field at a time.

With operator instructions. Add the top-level `system` field. Note it is a sibling of `messages`, and that on the 5-series it sits in front of a prompt where thinking is already on by default — so the same request costs more than the equivalent on a 4.5-generation model unless you say otherwise.

A tool-use continuation. Three messages: the user's question, the assistant turn containing thinking and `tool_use` blocks passed back **complete and unmodified** (Chapter 13's signature rule), and a user message containing **nothing but** `tool_result` blocks. Adding a note after the results ends the turn, and if a server tool was waiting, produces a 400.

A changed instruction mid-conversation. On Opus 5, Opus 4.8, Fable 5 or Mythos 5, append `{"role": "system", "content": "From now on, answer in French."}` after a user turn. Same authority as the

top-level field, and the cached prefix in front of it survives — which on a long conversation is the difference between section 16.21's two columns.

16.16 Code Walkthrough: The Protocol, Not the Wrapper

`ch16/wire.py` is 200 lines and has no dependencies. It exists to make the shape of section 16.5 unavoidable rather than optional.

The stop-reason table is data, not a chain of `if` statements.

```
STOP_REASONS = {
    "end_turn": ("Claude finished naturally", "use the response", True),
    "max_tokens": ("hit your max_tokens limit", "truncated: raise the "
                   "limit or continue the response", True),
    "stop_sequence": ("emitted one of your stop_sequences",
                      "read stop_sequence to see which fired", True),
    "tool_use": ("Claude is calling a tool",
                 "run it and return tool_result blocks", False),
    "pause_turn": ("a server-tool loop hit its iteration limit",
                   "send the assistant content back unchanged", False),
    "refusal": ("Claude declined", "read stop_details; retry on a "
                "fallback model", True),
    "model_context_window_exceeded": ("filled the context window",
                                      "treat as truncated", True),
}
```

The third element is the only bit of state a control loop needs: is the turn finished? Two of the seven are `False`. There is a test asserting exactly which two, so that if a future value arrives non-terminal the suite says so.

The correct read is a method, and the wrong one is a function kept for measurement.

```
def all_text(self):
    return "".join(b.get("text", "") for b in self.blocks("text"))
def naive_text(payload):
    """What most first drafts do. Included to be measured, not used."""
    return payload["content"][0]["text"]
```

Keeping the bug in the module as a named function is deliberate. Section 16.19 runs both against all seven stop reasons and counts, which turns "don't do that" into a number.

An unknown `stop_reason` raises.

```
sr = payload.get("stop_reason")
if sr not in STOP_REASONS:
    raise RequestError(f"unknown stop_reason {sr!r} — a new value "
                       f"has shipped; treat it as non-terminal and "
                       f"alert")
```

The alternative — falling through to "assume finished" — converts a future protocol addition into a silent data-loss bug. This is the same disposition as Chapter 14's registry raising on an unverified field: refuse to guess, loudly, early.

The request validator encodes the per-model rules.

```
if messages[-1]["role"] == "assistant" and model in NO_PREFILL:
    raise RequestError(
        f"assistant prefill returns 400 on {model}; prefilling is "
        f"unsupported on Claude 4.6 and later")
if m["role"] == "system":
    if i == 0:
        raise RequestError(
            "a system message cannot be first; use the top-level "
            "`system` field ...")
    if model not in MID_SYSTEM:
        raise RequestError(
            f"{model} does not accept mid-conversation system messages")
```

Two sets, `NO_PREFILL` and `MID_SYSTEM`, both from the documentation and both non-obvious: prefill is dead from the 4.6 generation onward, and mid-conversation system messages work on Fable 5, Mythos 5, Opus 4.8 and Opus 5 — *not* Sonnet 5. Chapter 14's rule again, in a different place.

`Conversation` **makes statelessness countable.**

```
class Conversation:
    def record_send(self):
        n = est_tokens(self.messages)
        if self.system:
            n += est_tokens(self.system)
        self.sent_input_tokens.append(n)
        return n
    @property
    def cumulative_input(self):
        return sum(self.sent_input_tokens)
```

It records what each turn actually puts on the wire, so `cumulative_input` is the real bill rather than the assumed one. The token estimate is four characters per token, labelled as an estimate in the docstring, with a pointer to Chapter 6's free and exact `count_tokens` endpoint. It is used here so the arithmetic runs offline; use the endpoint for anything that matters.

16.17 Mini Project: A Client Without an SDK

Goal. Build the request, parse the response, branch on `stop_reason`, and measure what statelessness costs — with no network, no key, and no library.

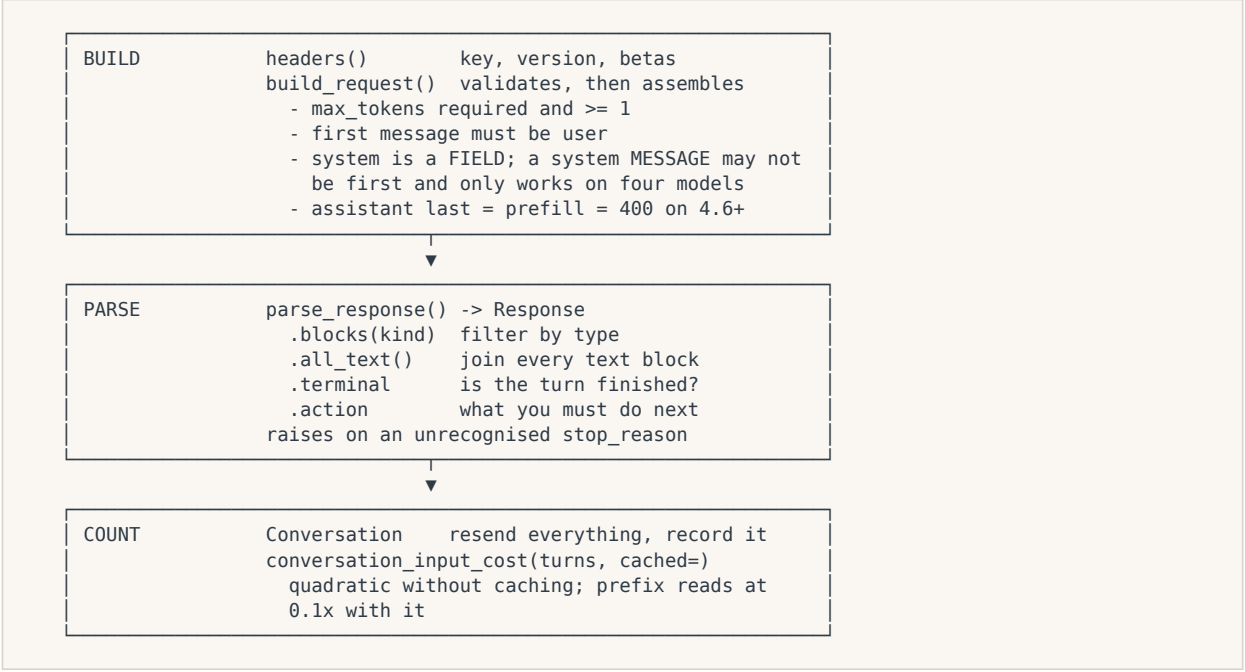


Figure 16.2 — wire.py: four responsibilities, one file.

```
ch16/
  wire.py         headers, build_request, parse_response, Conversation
  report.py       the six panels below
  test_wire.py    27 tests, 0.07 seconds
```

16.18 Panel A — The Wire

```
  POST https://api.anthropic.com/v1/messages
  x-api-key: sk-ant-...
  anthropic-version: 2023-06-01
  content-type: application/json
  {
    "model": "claude-opus-5",
    "max_tokens": 1024,
    "messages": [
      {
        "role": "user",
        "content": [
          { "type": "text", "text": "What is the capital of Peru?" }
        ]
      }
    ],
    "system": [
      { "type": "text", "text": "You answer in one sentence." }
    ],
    "output_config": { "effort": "low" }
  }
```

Two things this makes visual in a way prose does not. `system` **is a sibling of** `messages`, indented at the same level, not an entry inside the array. And `content` **is an array of typed blocks even for one line of text** — the string form you may pass is a convenience that expands to exactly this.

`output_config` is Chapter 13's effort knob, set to `low` here because the task is a lookup. On Opus 5 the default is `high`, and omitting the field is a decision to spend.

16.19 Panel B and C — The State Machine, and the Bug

```
B  THE STOP_REASON STATE MACHINE
  end_turn                        terminal
      use the response
  max_tokens                      terminal
      truncated: raise the limit or continue the response
  stop_sequence                   terminal
      read stop_sequence to see which fired
  tool_use                        NON-TERMINAL
      run it and return tool_result blocks
  pause_turn                      NON-TERMINAL
      send the assistant content back unchanged
  refusal                         terminal
      read stop_details; retry on a fallback model
  model_context_window_exceeded   terminal
      treat as truncated
```

Two of seven are non-terminal, and they fail differently from each other: `tool_use` waits on *your* code, `pause_turn` waits on the *server's* loop, which by default runs ten iterations per request. A handler that returns text on either one silently drops work.

Now the introduction's bug, run against every case:

```
C  WHAT content[0].text DOES TO EACH OF THEM
                     stop_reason        content[0].text             all_text()
                        end_turn    'The answer is 42.'    'The answer is 42.'
                      max_tokens        'The answer is'        'The answer is'
                    stop_sequence            'Part one.'            'Part one.'
                       tool_use              KeyError!                      ''
                     pause_turn              KeyError!                      ''
                        refusal           IndexError!                      ''
   model_context_window_exceeded      'As far as I got'      'As far as I got'
```

`content[0].text` **raises on three of the seven documented stop reasons.** On `tool_use` and `pause_turn` block zero is a tool-call block with no `text` key; on `refusal`, `content` is empty. Note also that the `tool_use` sample begins with a `thinking` block — which is the default on every 5-series model — so on those models the very first response your agent ever receives can break it.

The two rows where the naive read *succeeds* are arguably worse. `max_tokens` and `model_context_window_exceeded` both return a plausible-looking string that is a truncated answer, with no exception and no signal. That is the introduction's "oddly generic reply": a real answer, cut off, indistinguishable from a complete one unless you read `stop_reason`.

`all_text()` never raises. On the three broken rows it returns an empty string, which is correct — there *was* no text — and your code then branches on `stop_reason` and does the right thing.

16.20 Panel D — Statelessness Is Quadratic

Thirty turns of a realistic support conversation, counting what each request actually puts on the wire.

```
D  STATELESSNESS IS QUADRATIC
    turn  sent this turn   cumulative   naive x1   ratio
       1             69           69         69    1.0x
       2            189          258        138    1.9x
       5            548        1,542        345    4.5x
      10          1,147        6,078        690    8.8x
      20          2,344       24,130      1,380   17.5x
      30          3,542       54,158      2,070   26.2x
```

The `naive x1` column is the estimate teams actually make: "a turn is about seventy tokens, so thirty turns is about two thousand". The real figure is **54,158** — **26.2 times** larger.

The theory says the ratio should be `(n+1)/2`, or 15.5× at thirty turns. The measurement says 26.2×, and the gap is instructive rather than an error: the assistant turns in this simulation are longer than the user turns, so the history grows faster than the per-turn constant used in the naive estimate. Real conversations are worse still, because real assistant turns are much longer than real user turns.

Two design consequences, both of which Part 10 develops properly.

Turn count is a cost variable. Doubling the length of a conversation roughly quadruples its input bill. A product decision to "let the conversation run" is a budget decision.

By turn thirty, 96% of what you send is history. The newest turn is 3,542 tokens of which about 120 are new. You are paying full price to re-send text the server has already read twenty-nine times — which is the setup for the next panel.

16.21 Panel E — The Same Conversation, Cached

```
E  THE SAME CONVERSATION, WITH PROMPT CACHING
   turns   no cache $   cached $   saving
       5       0.0052     0.0025   51.7%
      10       0.0190     0.0059   69.1%
      20       0.0725     0.0152   79.0%
      30       0.1604     0.0279   82.6%
      50       0.4399     0.0638   85.5%
```

Chapter 11's multipliers applied to Chapter 1's statelessness: the stable prefix reads at **0.1×**, the new increment writes at **1.25×**. A conversation is close to the ideal shape for this, because everything except the newest turn is byte-identical to the previous request.

At thirty turns, caching removes 82.6% of the input cost, and the saving grows with length because the cacheable fraction grows. At a hundred thousand conversations a month on Opus 5 that is **$16,043 becoming $2,794**.

Three cautions that the table does not show.

Turn one is more expensive with caching, not less. There is no prefix to hit, so you pay 1.25× for the write. There is a test asserting it. Caching is a bet that the conversation continues, and on a workload of one-shot requests it is a straight 25% loss.

The five-minute default expires between turns. A user who takes six minutes to reply has cost you the whole prefix. Chapter 11 priced the one-hour duration at a 2× write multiplier; for human-paced conversation it is usually the right choice, and Chapter 30 covers the arithmetic.

Anything that changes the prefix invalidates it. Chapter 13's rule: the thinking configuration and the resolved `effort` level are rendered into the prompt, so varying either mid-conversation starts the cache over. This is also the reason section 16.3's mid-conversation system message exists — appending authority at the end preserves what caching has already bought.

16.22 Panel F — Six Requests the Validator Refuses to Send

```
F  SIX REQUESTS THE VALIDATOR REFUSES TO SEND
  system passed as a message
      a system message cannot be first; use the top-level `system`
      field for instructions that apply from the start
  assistant prefill on a 4.6+ model
      assistant prefill returns 400 on claude-opus-5; prefilling is
      unsupported on Claude 4.6 and later
  mid-conversation system on Sonnet 5
      claude-sonnet-5 does not accept mid-conversation system messages
  missing max_tokens
      max_tokens is required and must be >= 1
  conversation starting with the assistant
      the first message must have role 'user'
  a stop_reason nobody has seen before
      unknown stop_reason 'quota_exhausted' — a new value has shipped;
      treat it as non-terminal and alert
```

Five request errors and one response error, all caught locally in microseconds. Every one of them would otherwise cost a network round trip and produce an error message at the least convenient moment — in the first case, a 400 whose text does not obviously say "you came from a different vendor's API".

And the case that is accepted, because a validator that only ever says no is useless:

```
  A mid-conversation system message IS accepted on Opus 5:
  roles = ['user', 'assistant', 'system']
```

Same authority as the top-level field, appended at the end, cached prefix intact.

16.23 Evaluation and Deployment

Run `python3 report.py` and `python3 -m pytest` (27 tests, 0.07 seconds). Neither touches the network. To use the pattern in a real system:

1. **Keep the adapter; use the SDK for transport.** `build_request`, `parse_response` and the `stop_reason` dispatch are yours. Retries, backoff, streaming and connection reuse belong to the SDK — and to Chapter 29.
2. **Replace `est_tokens` with `count_tokens`.** Chapter 6's endpoint is free, exact, and rate-limited independently of the Messages API.

3. **Make the `stop_reason` dispatch exhaustive**, with the unknown case raising in development and alerting in production. Never fall through to "assume finished".
4. **Log `id`, `model`, `usage` and `stop_reason` on every response.** Four fields, zero cost, and every question in Chapters 15, 19 and 57 depends on them.
5. **Add a cache breakpoint at the end of the stable prefix** and measure `cache_read_input_tokens` against `cache_creation_input_tokens`. Panel E is the upper bound; your hit rate decides how much of it you get.
6. **Run Chapter 14's `preflight()` alongside `build_request()`.** This chapter's validator knows the protocol; that one knows the model.

16.24 Hands-on Labs

Lab 1 — Build one by hand. Make a Messages API request with `curl` and nothing else. *Success criterion:* a 200 with a `content` array, and you can name every header and field you sent.

Lab 2 — Break it deliberately. Send `system` as a message, prefill on a 5-series model, omit `max_tokens`, set `temperature: 0.5` on Opus 5. *Success criterion:* four distinct 400s, and you can match each error message to its cause.

Lab 3 — Audit your response handling. Find every place your codebase reads a response. *Success criterion:* zero uses of `content[0]`, and an exhaustive `stop_reason` branch with a loud default.

Lab 4 — Measure your own conversation growth. Instrument a real multi-turn feature and record `input_tokens` per turn. *Success criterion:* a chart of cumulative input against turn number, and the ratio against your naive estimate.

Lab 5 — Cache it. Add a breakpoint at the end of the stable prefix and re-measure. *Success criterion:* a measured hit rate and a monthly saving, compared against Panel E's ceiling.

16.25 Lab Solutions

Lab 1. Three headers, three required fields. If it fails, strip to the minimum — `model`, `max_tokens`, one user message — and add fields one at a time; the field you add when it breaks is the field that is wrong.

Lab 2. The `system`-as-message case is the one worth dwelling on: the error names the field but not the reason, and a developer arriving from another vendor's API will read it three times. That is precisely why the validator's message says *use the top-level* `system` *field* rather than merely "invalid role".

Lab 3. Grep for `content[0]`, `content[-1]`, and `.text` without a type check. The `stop_reason` branch should be a dictionary lookup or a match statement covering all seven, with the default raising rather than returning. Add a test that feeds a synthetic unknown value.

Lab 4. `usage.input_tokens` per turn is the exact figure; do not estimate. Plot the cumulative sum. Expect a ratio in the tens by turn twenty. If your ratio is close to 1, you are truncating history somewhere and should find out where before something depends on it.

Lab 5. Put the breakpoint at the end of everything that does not change: system prompt, tool definitions, and any fixed preamble. Watch `cache_read_input_tokens` in `usage`. A hit rate below 50% on a

conversational workload usually means the prefix is not stable — the commonest cause is a timestamp or a session ID interpolated into the system prompt, which is Chapter 11's misplaced-prefix failure in miniature.

16.26 Best Practices

BEST PRACTICE

Using the Messages API without surprises

1. **Set `anthropic-version`.** It is required and it pins the contract.
2. **Put operator instructions in the top-level `system` field**, and mid-conversation instructions in a system message where the model supports it.
3. **Never index `content` by position.** Filter by `type`.
4. **Branch exhaustively on `stop_reason`**, and raise or alert on an unrecognised value.
5. **Know which two are non-terminal.** `tool_use` waits on you; `pause_turn` waits on the server.
6. **Treat a refusal as a 200 with a decision in it**, not as an outage.
7. **Log `id`, `model`, `usage` and `stop_reason` on every response.**
8. **Set `max_tokens` deliberately.** It is the only enforced ceiling, and it includes thinking tokens.
9. **Validate before sending.** Local checks cost microseconds; 400s cost round trips and confusion.
10. **Remember the conversation is a list you resend**, and that its cost is quadratic in turns.
11. **Cache the stable prefix**, and keep it genuinely stable — no timestamps, no session IDs.
12. **Use `count_tokens` rather than estimating.** Free, exact, separate rate limits.
13. **Stop using prefill.** It is a 400 from the 4.6 generation onward; use structured outputs.
14. **Keep protocol knowledge in one adapter**, so a new stop reason or block type is one edit.

16.27 Common Mistakes

COMMON MISTAKE

Reading `content[0].text`

Symptom. Intermittent `KeyError` or `IndexError`, or replies that are plausible but truncated, on a fraction of traffic. **Cause.** `content` is an array of typed blocks. Block zero is a `thinking` block on every 5-series model by default, a `tool_use` block whenever a tool is called, and absent on a refusal. **Fix.** Join every block whose `type` is `text`, and branch on `stop_reason` for everything else. **Detection.** Grep for `content[0]`. Section 16.19 measured this breaking on three of seven documented stop reasons.

COMMON MISTAKE

Passing `system` as a message

Symptom. A 400 on the first request, usually while porting from another vendor's API. **Cause.** `system` is a top-level field. A `system` *message* is a different, newer feature that may not come first and works on only four models. **Fix.** Top-level `system` for instructions from the start; a mid-conversation system message on Fable 5, Mythos 5, Opus 4.8 or Opus 5 for instructions that arrive later. **Detection.** Validate locally; the validator's message can say what the API's does not.

COMMON MISTAKE

Treating `stop_reason` as a log field

Symptom. An agent that silently drops tool calls, or a document pipeline that emits truncated output nobody notices. **Cause.** Ignoring the branch. `tool_use` and `pause_turn` mean the turn is unfinished; `max_tokens` and `model_context_window_exceeded` mean the content is incomplete but present. **Fix.** An exhaustive dispatch with a loud default. **Detection.** Count `stop_reason` values in your logs. If you cannot, that is the finding.

COMMON MISTAKE

Alerting on refusals as errors

Symptom. Pager noise from HTTP 200 responses, or conversely a silent quality drop nobody notices. **Cause.** A refusal is a successful request whose answer is "no", carrying `stop_details` with the policy category. It is not a 4xx. **Fix.** Treat it as a branch: read the category, retry on a fallback model, and track the rate as a quality metric. **Detection.** Check whether your error rate and your refusal rate are the same number. They should not be.

COMMON MISTAKE

Budgeting a conversation as if it were linear

Symptom. Conversational costs several times the forecast, with no bug to point at. **Cause.** Statelessness. Turn `i` resends `i` turns of history, so the total is quadratic. Section 16.20 measured 26.2× at thirty turns against the naive estimate. **Fix.** Model the sum, not the turn. Then cache the prefix, and compact when the conversation gets long (Chapter 54). **Detection.** Plot cumulative `input_tokens` against turn number. A straight line means you are truncating history somewhere.

COMMON MISTAKE

Assuming a stable cache prefix that is not stable

Symptom. `cache_read_input_tokens` near zero despite a cache breakpoint being set. **Cause.** Something varies in the prefix — a timestamp, a session ID, a user name, or a changed `effort` or thinking configuration, both of which are rendered into the prompt. **Fix.** Move variable content behind the breakpoint. Hold effort and thinking constant for the life of the conversation. **Detection.** Diff two consecutive rendered prefixes byte for byte. This is Chapter 11's misplaced-prefix failure, which cost +20.6% against no caching at all.

COMMON MISTAKE

Carrying prefill forward from an old integration

Symptom. Every request 400s after a model upgrade, on requests that end with an assistant message. **Cause.** Prefilling is unsupported on Claude 4.6 and later and on Mythos Preview. **Fix.** Structured outputs where available (Chapter 10 and Chapter 23), or system prompt instructions. **Detection.** Search for requests whose last message has `"role": "assistant"`. The validator in section 16.22 catches them.

16.28 Performance Considerations

The request shape has three levers on latency and they are all about tokens. `max_tokens` bounds the worst case but does not shorten the typical response. `effort` moves the actual output token count, which is the term that dominates Chapter 11's model. And thinking, from Chapter 13, can multiply the decode term seventeen-fold on a short answer — so the biggest performance decision in a Messages API request is frequently the `output_config` field rather than anything about the messages.

On the input side, the quadratic growth of section 16.20 is a *prefill* cost rather than a decode cost. Chapter 4 established that prefill parallelises across the sequence, so a long history costs less in wall clock than its token count suggests — but it costs full price in money, and prompt caching accelerates prefill specifically. Section 16.21's 82.6% is a cost saving with a latency saving alongside it.

Two client-side facts worth carrying. The SDKs **require streaming when** `max_tokens` **exceeds 21,333**, which is a client-side guard rather than an API restriction. And Chapter 13's advice to move thinking-heavy work above roughly 32,000 thinking tokens onto the Batch API applies here as a transport decision: long-running synchronous requests hit system timeouts and open-connection limits.

16.29 Security Considerations

The message history is entirely under your control, and the model cannot tell. Synthetic assistant turns are a documented and useful technique, which means the trust boundary is at your application, not at the API. Anything that lets an untrusted party influence the messages array — including tool results, retrieved documents and user-supplied content — is an injection surface. Chapter 58 covers it properly; the shape-level point is that "assistant said this" is an assertion your code makes, not a fact the API verifies.

The system prompt is where operator authority lives, and it is not a secret. Instructions in the system field carry weight, and users will attempt to elicit them. Put authority there; do not put credentials, keys or confidential policy text there.

Thinking blocks must be round-tripped unmodified. Chapter 13's signature rule is a request-shape rule: within a tool-use turn, pass `thinking` and `redacted_thinking` blocks back complete and unchanged, or the request is rejected with a 400. Code that filters blocks by `type == "thinking"` alone silently drops `redacted_thinking` and breaks the protocol.

`usage` **is a side channel.** Token counts leak information about content length and structure. If you expose response metadata to end users — in an API you resell, for instance — consider whether the token counts tell them something the response body does not.

Log deliberately. `id`, `model`, `usage` and `stop_reason` are safe and essential. Message content is the sensitive part, and Chapter 61 covers the retention obligations that attach to it once you have written it to a log.

16.30 Real Production Examples

The chat product. The archetype for section 16.20. Cost is dominated by resent history, so the levers are prompt caching on the stable prefix, a one-hour cache duration to survive human reply latency, and compaction once the conversation gets long. The mistake to avoid is interpolating anything variable into the system prompt.

The batch classifier. No conversation, no history, one turn each. Caching the system prompt and few-shot examples across requests is the whole optimisation, and the Batch API halves it again. Note that turn one is *dearer* with caching, so on a workload of genuinely unique single requests the cache is a 25% loss.

The tool-using agent. Lives or dies on the `stop_reason` branch. `tool_use` requires a user message of nothing but `tool_result` blocks; `pause_turn` requires the assistant content sent back unchanged; thinking blocks must be round-tripped with their signatures. This is the loop the introduction's bug was inside, and Part 6 builds it correctly.

The document pipeline. Long outputs, so `max_tokens` and `model_context_window_exceeded` are live concerns rather than theoretical ones. Both mean truncation, and the documented practice is to append a notice so the reader knows the output is incomplete — or to continue generation from where it stopped, which needs the `stop_reason` branch to distinguish "finished" from "ran out".

16.31 Certification Notes

Concept	Exam domain	What is likely to be asked
Endpoint and headers	D5 Context Management & Reliability	`POST /v1/messages`; `x-api-key`, `anthropic-version: 2023-06-01`
Required fields	D4 Prompt Engineering & Structured Output	`model`, `max_tokens`, `messages` — all three
`system` as a field	D4	Top-level, not a message; the mid-conversation exception and its four models
Typed content blocks	D4, D1	Never index by position; filter by `type`
The seven stop reasons	D1 Agentic Architecture & Orchestration	Which two are non-terminal, and what each requires
Stop reasons vs errors	D5	A refusal is an HTTP 200
`pause_turn`	D1, D2	Server-tool loop limit, default 10; send the content back unchanged
Statelessness	D5	Full history every request; quadratic cost

Concept	Exam domain	What is likely to be asked
Prefill	D4	400 on Claude 4.6 and later; use structured outputs
Sampling parameters	D4	`temperature`/`top_p`/`top_k` 400 from Opus 4.7 onward
`usage`	D5	`input_tokens`, `output_tokens`, cache fields, `thinking_tokens`
Messages API vs Managed Agents	D1	Custom loops versus a managed harness for long-running work

CERTIFICATION NOTE

Exam Tips

- **Three required fields:** `model`, `max_tokens`, `messages`. `max_tokens` being required catches people.
- `system` **is a top-level field.** Any option showing it as `messages[0]` is wrong — unless the question is specifically about mid-conversation system messages, which cannot be first and work on four models.
- `tool_use` **and** `pause_turn` **are the two non-terminal stop reasons**, and they are continued differently: tool results versus the content sent back unchanged.
- **A refusal is a 200 with** `stop_details`. If a question frames it as an error, that framing is the trap.
- `model_context_window_exceeded` means you hit the context window, not `max_tokens`.
- **Statelessness means you send the whole history every time.** Any option implying server-side session state is wrong.
- **Prefill returns a 400 on 4.6 and later.** The replacement is structured outputs.
- `content` **is always an array.** Watch for options that treat it as a string.

16.32 Twenty Practice Questions

1. What are the three required fields in a Messages API request?
2. Which three headers does every request need, and what does the version header pin?
3. Where does the system prompt go, and what is the one exception?
4. Which models accept a mid-conversation system message, and why does the feature exist?
5. Why is `content[0].text` a bug?
6. Name all seven `stop_reason` values.
7. Which two are non-terminal, and how is each one continued?
8. What is the difference between `tool_use` and `pause_turn`?
9. What HTTP status does a refusal return, and what field carries the reason?
10. What does `model_context_window_exceeded` mean, and how does it differ from `max_tokens`?
11. What must the continuation message after `tool_use` contain, and what must it not?
12. Why is a conversation's input cost quadratic in the number of turns?
13. A 30-turn conversation: how does the real input token count compare with the naive estimate?

14. What does prompt caching do to that cost, and why does it fit conversations so well?
15. Why is the first turn of a cached conversation more expensive, not less?
16. What happened to prefilling, and what replaced it?
17. Which sampling parameters return a 400, and on which models?
18. What is in `usage`, and why should you log it?
19. What are the three image source types, and which media types are supported?
20. When would you choose Claude Managed Agents over the Messages API?

16.33 Answers and Explanations

1. `model`, `max_tokens` **and** `messages`. `max_tokens` *being required is the one people miss; it has no default because there is no safe default for a hard spend ceiling.*
2. `x-api-key`, `anthropic-version` **and** `content-type`**.** The version header pins the **API contract**, not the model — it has been `2023-06-01` throughout. *A fourth,* `anthropic-beta`*, appears for preview features.*
3. In the **top-level** `system` **field**, a sibling of `messages`. The exception is a **mid-conversation system message**, which may not be first. *Passing* `system` *as* `messages[0]` *is a 400 and the commonest port-from-another-vendor error.*
4. **Fable 5, Mythos 5, Opus 4.8 and Opus 5.** It exists for caching: the top-level field sits at the front of the prompt, so changing it invalidates every cached prefix behind it, whereas a system message is appended at the end and invalidates nothing. *Sonnet 5 does not support it.*
5. Because `content` is an **array of typed blocks**. Block zero is a `thinking` block on every 5-series model by default, a `tool_use` block when a tool is called, and absent on a refusal — so the read raises on three of seven stop reasons and silently returns a partial answer on two more. *Filter by* `type` *instead.*
6. `end_turn`, `max_tokens`, `stop_sequence`, `tool_use`, `pause_turn`, `refusal`, `model_context_window_exceeded`**.** *An eighth will ship; handle the unknown case loudly.*
7. `tool_use` — continue with a user message of `tool_result` blocks. `pause_turn` — continue by sending the assistant content back unchanged. *Both mean the assistant turn is unfinished.*
8. `tool_use` means Claude is waiting on **your** code. `pause_turn` means the **server-side** sampling loop hit its iteration limit while running server tools, default **10 per request**. *A response waiting on your tools never has* `stop_reason: "pause_turn"`.
9. **HTTP 200** — it is a successful request. `stop_details` carries the policy category, and is `null` for every other stop reason. *The documented remedy is a retry on a fallback model.*
10. It means generation filled the **model's context window** rather than your `max_tokens`. It lets you request maximum output without computing input size first. *Typed in the SDKs' beta namespace; returned without a header on Sonnet 4.5 and newer.*
11. **Nothing but** `tool_result` **blocks**, one per `tool_use` block. Adding anything after them — text, for instance — ends the assistant turn, and if a server tool was left waiting the request fails with a 400 naming the unresolved tool. *To add input, send a separate user message after the turn completes.*
12. Because the API is **stateless**: turn `i` resends all `i` turns of history, so the total is `m·n(n+1)/2`. *There is no server-side session to append to.*

13. In section 16.20's measurement, **54,158 tokens against a naive 2,070 — a factor of 26.2**. The theoretical `(n+1)/2` gives 15.5×; the excess comes from assistant turns being longer than user turns. *Real conversations are worse.*
14. Caching reads the stable prefix at **0.1×**, saving **82.6% at thirty turns** and more as the conversation grows. It fits because a conversation is almost entirely prefix — everything except the newest turn is byte-identical to the previous request. *Caching is prefix-only, which is exactly this shape.*
15. There is **no prefix to hit**, so you pay the **1.25×** write multiplier for a five-minute cache. Caching is a bet the conversation continues; on genuinely one-shot requests it is a 25% loss. *There is a test asserting this.*
16. **Prefilling returns a 400 on Claude 4.6 and later and on Mythos Preview.** The documented replacements are **structured outputs** and **system prompt instructions**. *This is stronger than the older "you cannot prefill while thinking is on".*
17. `temperature`, `top_p` **and** `top_k`, on **Claude 4.7 and later** and Mythos Preview, when set to a non-default value. *They remain in the SDK request types, so this fails at runtime rather than compile time.*
18. `input_tokens` **and** `output_tokens`, plus the cache fields and `output_tokens_details.thinking_tokens`. Log it because it is your bill arriving with every response, and every cost question in Chapters 15 and 19 depends on having it. *It is free and it is gone if you do not capture it.*
19. `base64`, `url` **and** `file` — the last referencing a Files API upload. Supported media types: `image/jpeg`, `image/png`, `image/gif`, `image/webp`. *Chapter 17 covers the token cost and quality questions.*
20. When the task is **long-running or asynchronous** and the agent loop is undifferentiated work. The Messages API is for **custom agent loops and fine-grained control**, and choosing it means owning the retries, the tool execution, the conversation state and the observability. *Both are documented options; the choice is about what you want to own.*

16.34 Ten Practical Exercises

1. Add streaming to `wire.py`: parse the SSE event sequence and reconstruct the final message. Compare against Chapter 17.
2. Add a `tool_result` builder that enforces the "nothing but tool results" rule and the matching-ids rule.
3. Extend `Conversation` with a compaction strategy: summarise turns older than `k` and measure the cost curve's new shape.
4. Add a cache-breakpoint model to `conversation_input_cost` that expires the prefix after five minutes of simulated user think-time.
5. Implement the `max_tokens` continuation pattern: detect truncation, resend with the partial response, and stitch.
6. Add `metadata` and use it to attribute cost per tenant from `usage` alone.
7. Write a fuzzer that generates malformed responses and assert `parse_response` never returns silently wrong data.
8. Add a `pause_turn` loop with a continuation cap, and test that it terminates.

9. Model the server-tool detection rule: find `server_tool_use` ids with no matching result block.
10. Implement `count_tokens` against the real endpoint and measure how far the four-characters-per-token estimate is off on your own traffic.

16.35 Five Challenge Labs

1. **A complete stateless agent loop.** Tools, thinking blocks round-tripped with signatures, all seven stop reasons handled, caching, and a hard turn cap. Then prove termination for every branch.
2. **The cost model that matches the invoice.** Instrument a real conversational product, model its cost from `usage` alone, and reconcile against the actual bill to within 2%. Report what you had to add to the model to get there.
3. **Protocol drift detection.** Record every distinct response shape your system sees for a month — block types, stop reasons, usage fields — and alert on anything new. Report what appeared.
4. **The adapter migration.** Take a codebase with model strings and response handling scattered across it and move both behind Chapter 14's registry and this chapter's adapter, with no behaviour change. Measure the diff size and the number of latent bugs found.
5. **Caching under real user latency.** Instrument reply gaps in a live conversation, model cache expiry against the five-minute and one-hour durations, and compute which is cheaper for your distribution. Report the crossover gap length.

16.36 Ten Interview Questions

1. Walk me through a Messages API request from headers to response.
2. Why is `system` a top-level field rather than a message?
3. What is wrong with `response.content[0].text`?
4. Name the stop reasons that mean the turn is not finished, and how each is continued.
5. A refusal comes back. Is that an error? What do you do?
6. Why does a long conversation cost more than the sum of its turns?
7. When does prompt caching lose you money?
8. What did prefill do, why is it gone, and what replaced it?
9. What would you log from every response, and why those fields?
10. When would you not use the Messages API at all?

16.37 Summary

Everything in Parts 5 through 12 of this book goes through one HTTP request: `POST /v1/messages`, with three headers and three required fields. The transport is unremarkable and the body is where the design lives. `model` names a pinned snapshot from Chapter 14; `max_tokens` is the only enforced ceiling in the entire configuration surface, counting thinking tokens alongside visible text; and `messages` is the conversation, which you own and resend in full. Everything else — `system`, `tools`, `thinking`, `output_config`, `stop_sequences`, `stream` — is optional, which makes debugging tractable: strip to the

three required fields and add one at a time.

Three shape facts carry most of the chapter's practical weight, and all three are running threads of the book arriving at the place they are enforced. The system prompt is a **top-level field**, a sibling of `messages`, because operator authority is a property of the request rather than a turn in the dialogue — with one 2026 refinement worth knowing precisely: Fable 5, Mythos 5, Opus 4.8 and Opus 5 accept a mid-conversation `system` message after a user turn, carrying the same authority, and it exists for a caching reason, since the top-level field sits at the front of the prompt and invalidates every cached prefix behind it while an appended message invalidates nothing. `content` is an **array of typed blocks** on both sides of the wire, so `content[0].text` is a bug rather than a shortcut: measured against every documented stop reason it raises on three of seven, and on two more it returns a truncated answer that looks complete. And `stop_reason` is a **control-flow branch**, seven values of which two — `tool_use` and `pause_turn` — mean the assistant turn is unfinished and your loop owes the API another request, continued differently from each other, one with tool results and one with the assistant content sent back unchanged.

Stop reasons are not errors, and conflating them produces noise in one direction and silence in the other. A refusal arrives as an HTTP 200 carrying `stop_details` with a policy category; a 429 is a failed request. `max_tokens` and `model_context_window_exceeded` are both successful requests whose content is incomplete, which is precisely the failure that looks like nothing at all in a dashboard. The chapter's advice is to make the dispatch exhaustive and to raise on an unrecognised value, because an eighth stop reason will ship and a default branch that assumes "finished" converts that future addition into silent data loss.

Statelessness is the running thread from Chapter 1 and this is where it is finally priced. The server keeps nothing, so turn `i` carries `i` turns of history and the total input over `n` turns is `m·n(n+1)/2` — quadratic. The naive estimate, that thirty turns of seventy tokens is about two thousand tokens, understates the measurement by a factor of **26.2**, partly from the arithmetic and partly because assistant turns are longer than user turns. By turn thirty, 96% of every request is text the server has already read. That shape is unusually well suited to prompt caching, because a conversation is almost entirely prefix, and Chapter 11's 0.1× read multiplier removes **82.6% of the input cost at thirty turns**, rising with length — $16,043 a month becoming $2,794 at a hundred thousand conversations. With three cautions: turn one is *dearer* with caching because there is no prefix to hit, the five-minute default expires between human replies, and anything that varies in the prefix — a timestamp, a session ID, a changed `effort` level — destroys the whole saving.

Two things have changed and one choice remains. Prefilling is dead: a 400 on Claude 4.6 and later, replaced by structured outputs, which is a genuine upgrade from a hack that worked to a mechanism that holds. Sampling parameters are dead from Opus 4.7 onward, returning 400 on non-default values while remaining in the SDK request types, which is why they fail at runtime and never at compile time. And the standing choice is whether to use this endpoint at all: the Messages API gives you the loop and the fine-grained control, and Claude Managed Agents give you a harness for long-running asynchronous work. This book teaches the loop, because you cannot architect one you have never held — but building it is a decision with a cost, and the right place for the knowledge is a single adapter with the protocol in it, not fifty call sites that each learned a little of it.

KEY TAKEAWAYS

1. **One endpoint, three headers, three required fields**: `model`, `max_tokens`, `messages`.
2. `system` **is a top-level field**; a mid-conversation system message is a different, newer thing on four models.
3. `content` **is an array of typed blocks.** `content[0].text` raises on three of seven stop reasons.
4. **Seven stop reasons;** `tool_use` **and** `pause_turn` **are non-terminal** and are continued differently.
5. **A refusal is an HTTP 200** with `stop_details`. Stop reasons are not errors.
6. `max_tokens` **is the only enforced ceiling**, and it counts thinking tokens.
7. **Statelessness is quadratic**: 54,158 tokens over 30 turns against a naive 2,070 — 26.2×.
8. **Caching removes 82.6% of that at 30 turns**, and costs you 25% on turn one.
9. **Prefill returns 400 from Claude 4.6 onward**; structured outputs replaced it.
10. **Log** `id`, `model`, `usage` **and** `stop_reason` **on every response.** Four fields, zero cost.

16.38 Glossary

Term	Definition
`anthropic-beta`	Header carrying a comma-separated list of preview feature flags.
`anthropic-version`	Required header pinning the API contract; `2023-06-01`. Not the model version.
Content block	A typed element of `content`: `text`, `thinking`, `tool_use`, `image`, and others.
`max_tokens`	Required hard ceiling on generated tokens, thinking included. The only enforced limit.
Mid-conversation system message	A `role: "system"` message after a user turn; four models; may not be first.
`pause_turn`	Non-terminal stop reason: the server-tool loop hit its iteration limit, default 10.
Prefill	Putting the start of the assistant's reply in the last message. A 400 on 4.6 and later.
Refusal	A stop reason, returned as HTTP 200, with the policy category in `stop_details`.
Statelessness	The server retains nothing between requests; the full history is resent each time.
`stop_details`	Present only on `refusal`; identifies the policy category. `null` otherwise.
`stop_reason`	Why generation ended. Seven documented values; two are non-terminal.
Synthetic assistant message	An assistant turn your code wrote rather than the model. Documented and allowed.
System prompt	Top-level `system` field carrying operator authority.
`usage`	Per-response token accounting: input, output, cache fields, thinking tokens.

16.39 Further Reading

Primary sources. `platform.claude.com/docs/en/build-with-claude/working-with-messages` for the request and response shape, multi-turn patterns, synthetic assistant messages, the prefill deprecation and the vision source types; `.../build-with-claude/handling-stop-reasons` for all seven stop reasons with handling code, the tool-result continuation rules and the stop-reasons-versus-errors distinction; `.../build-with-claude/mid-conversation-system-messages` for the placement rules and the caching interaction; `.../api/messages/create` for the complete field reference; and `.../build-with-claude/refusals-and-fallback` for the refusal categories and the fallback pattern. `.../managed-agents/overview` is the alternative to building the loop yourself.

Where to go next in this book. Chapter 17 is the direct sequel: streaming, vision, files and documents — the input modalities and the server-sent-event form of everything in this chapter. Chapter 18 is thinking as an API surface, expanding the `thinking` and `output_config` fields into working code. Chapter 19 covers rate limits and quotas, which is where `usage` becomes an operational constraint. Chapter 23 builds JSON contracts on the structured outputs that replaced prefill. Chapters 26 to 30 are the API in depth: authentication and SDKs, request parameters, response handling, errors and retries, and batch processing with prompt caching — Chapter 30 being where section 16.21's caching arithmetic is built properly. Part 6 turns the `tool_use` branch into a working loop, Chapter 54 covers the compaction that answers section 16.20's quadratic curve, and Chapter 57 is the observability that makes `id`, `model`, `usage` and `stop_reason` into something you can answer questions with.